

— Databases

Hoofdstuk 8: Schrijfqueries

L. Espeel



— Schrijfqueries

In hoofdstuk 3: reeds deze DDL- en DML-queries gezien:

- CREATE DATABASE
- DROP DATABASE
- CREATE TABLE
- DROP TABLE
- INSERT

In dit hoofdstuk: nog enkele DDL- en DML-queries worden behandeld:

- ALTER TABLE
- TRUNCATE TABLE
- UPDATE
- DELETE

— ALTER TABLE ⁽¹⁾

Om aanpassingen te doen aan de structuur van een bestaande tabel waarin vaak al gegevens staan (!), is het af te raden om de tabel eerst te verwijderen (DROP TABLE) en vervolgens opnieuw aan te maken (CREATE TABLE) en de gegevens opnieuw toe te voegen (INSERT).

Om wijzigingen aan te brengen in de structuur gebruiken we ALTER TABLE.

Enkele voorbeelden:

- Kolom toevoegen

```
ALTER TABLE inschrijvingen  
ADD COLUMN inschrijvingstijd TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
```

Opmerking: door een default waarde te voorzien kunnen we de reeds bestaande records in de tabel hiermee opvullen.

- Kolom (terug) verwijderen

```
ALTER TABLE inschrijvingen  
DROP COLUMN inschrijvingstijd;
```

3

— ALTER TABLE ⁽²⁾

- Naam van een kolom wijzigen

```
ALTER TABLE inschrijvingen  
CHANGE COLUMN inschrijvingstijd inschrijvingsmoment TIMESTAMP;
```

De oude- en nieuwe kolomnaam moeten steeds opgegeven worden.

- Datatype van een kolom wijzigen

```
ALTER TABLE inschrijvingen  
CHANGE COLUMN inschrijvingsmoment inschrijvingsmoment DATETIME;
```

Merk op dat hier de oude- en nieuwe kolomnaam dezelfde naam hebben.
Er bestaat ook een verkorte vorm om enkel het datatype te veranderen:

```
ALTER TABLE inschrijvingen  
MODIFY inschrijvingsmoment DATETIME;
```

4

— UPDATE⁽¹⁾

Met een update-query kunnen we één of meerdere velden aanpassen.

- **Advies:** schrijf voor de veiligheid eerst een SELECT-query met de juiste WHERE-conditie en test uit. Vorm de query daarna om tot een UPDATE-query.

Enkele voorbeelden:

- **UPDATE leden**
SET adres = "Dreef 1", postnummer = 9810, gemeente = "Nazareth"
WHERE lidnr = 1;

5

— UPDATE⁽²⁾

- **UPDATE activiteiten**
SET datumtijdstip = ADDTIME(datumtijdstip, "00:01:00")
WHERE omschrijving LIKE "%nacht%";

Met **ADDTIME** kunnen we een tijdsinterval instellen die we willen bijtellen.
Negatieve waarden kunnen uiteraard ook.

- **UPDATE activiteiten**
SET datumtijdstip = ADDDATE(datumtijdstip, INTERVAL 3 DAY);

Met **ADDDATE** kunnen we een datuminterval (dag, maand of jaar) instellen die we willen bijtellen.
Negatieve waarden kunnen uiteraard ook.

6

— DELETE ⁽¹⁾

Met een delete-query kunnen we één of meerdere records verwijderen.

- **Advies:** schrijf voor de veiligheid eerst een SELECT-query met de juiste WHERE-conditie en test uit. Vorm de query daarna om tot een DELETE-query.

Enkele voorbeelden:

- DELETE
FROM leden
WHERE gemeente = "Waregem";
- DELETE
FROM activiteiten
WHERE omschrijving LIKE "%skikamp%"
ORDER BY datumtijdstip
LIMIT 1;

Verwijder het oudste skikamp.

7

— DELETE ⁽²⁾

- DELETE
FROM activiteiten
WHERE omschrijving = "Halloweentocht";

Dit zal niet lukken en levert deze foutmelding op:

*Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails
(`jeugdbeweging`.`inschrijvingen`, CONSTRAINT `fk2` FOREIGN KEY (`activiteit\_id`) REFERENCES
`activiteiten` (`activiteit\_id`))*

Reden: in de tabel *inschrijvingen* staan nog records die via hun externe sleutel (*activiteit_id* = 2 of *activiteit_id* = 11) naar de activiteit "Halloweentocht" verwijzen.

Zolang deze inschrijvingen bestaan, verhindert de referentiële integriteit dat de bijbehorende activiteit wordt verwijderd.

(Bv. met activiteit "Skikamp" zou het daarentegen wel lukken om direct te wissen.)

8

— DELETE (3)

Hoe toch wissen?

Dit moet hier in twee stappen gebeuren:

1. Verwijder eerst de records uit de tabel *inschrijvingen* waarbij *activiteit_id* = 2 of 11.
2. Verwijder daarna de records uit de tabel *activiteiten* waarbij de omschrijving "Halloweentocht" is.

De elegantste manier om zonder hardcoded id-waarden te werken, is door in de eerste stap al gebruik te maken van een **subquery** of **join**:

```
DELETE
FROM inschrijvingen
WHERE activiteit_id IN
(
    SELECT activiteit_id
    FROM activiteiten
    WHERE omschrijving = "Halloweentocht"
);
```

```
DELETE
FROM activiteiten
WHERE omschrijving = "Halloweentocht";
```

9

— DELETE (4)

```
DELETE i
FROM inschrijvingen i
JOIN activiteiten a ON i.activiteit_id = a.activiteit_id
WHERE a.omschrijving = "Halloweentocht";
```

```
DELETE
FROM activiteiten
WHERE omschrijving = "Halloweentocht";
```

10

— TRUNCATE TABLE

Is een alternatief voor DELETE om in één keer alle records te wissen. Dit gebeurt bovendien sneller dan DELETE en de auto-incrementwaarde (van de primaire sleutel) wordt daarbij terug op 0 gezet zodat een nadien ingevoegd record opnieuw de waarde 1 krijgt.

Maar: dit kan niet worden gebruikt wanneer andere tabellen via externe sleutels naar deze tabel verwijzen. Daardoor is het in de praktijk vaak niet bruikbaar.

```
TRUNCATE TABLE inschrijvingen;
```

11

— MySQL Workbench query-beveiliging ⁽¹⁾

Problemen:

- MySQL Workbench bevat standaard een beveiligingsmechanisme dat voorkomt dat schrijfqueries zonder WHERE-conditie worden uitgevoerd:
`DELETE FROM inschrijvingen;`

```
UPDATE activiteiten  
SET omschrijving = "Kerstfeestje";
```

- Ook laat MySQL Workbench het uitvoeren van de query niet toe als de WHERE-conditie een string-veld bevat dat geen index is:
`UPDATE leden
SET naam = "Iris De Smet"
WHERE naam = "Iris Desmet";`

12

MySQL Workbench query-beveiliging (2)

Oplossing: zet deze beveiliging af in MySQL Workbench, dit moet maar éénmalig gebeuren:

- Ga naar menu *Edit > Preferences... > SQL Editor > vinkje afzetten onderaan (scrollen!) bij Safe Updates (rejects UPDATES AND DELETES with no restrictions) > OK*
- Om deze wijzigingen (zonder herstarten van MySQL Workbench) door te voeren, ga dan naar menu *Query > Reconnect to server*.

